

Deep Learning Laboratory Manual

AY 2025-26

1. Basic Neural Networks

Objective: Implement basic neural network components.**Tasks:** Write a Python program to create and train a simple neural network from scratch using NumPy. This includes implementing models of neurons, activation functions, and basic network architecture.

```

# Import necessary libraries
import tensorflow as tf from
tensorflow import keras
from tensorflow.keras import
layers import numpy as np
import matplotlib.pyplot as plt

# Load and preprocess MNIST dataset
(X_train, y_train), (X_test, y_test) = keras.datasets.mnist.load_data()
X_train, X_test = X_train / 255.0, X_test / 255.0 # Normalize data
X_train = X_train.reshape(-1, 28 * 28) # Flatten images
X_test = X_test.reshape(-1, 28 * 28)
y_train = keras.utils.to_categorical(y_train, 10) # One-hot encode labels
y_test = keras.utils.to_categorical(y_test, 10)

# Define the neural network
model = keras.Sequential([
    layers.Dense(128, activation='relu', input_shape=(784,)),
    layers.Dense(64, activation='relu'), layers.Dense(10,
    activation='softmax') ])

# Compile the model
model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])

# Train the model
history = model.fit(X_train, y_train, epochs=10, batch_size=32, validation_split=0.2)

# Evaluate on test data
test_loss, test_acc = model.evaluate(X_test, y_test)
print(f"Test Accuracy: {test_acc:.4f}")

# Make predictions
predictions = model.predict(X_test)
predicted_label = np.argmax(predictions[0])
actual_label = np.argmax(y_test[0])

# Display an image with its predicted label
plt.imshow(X_test[0].reshape(28, 28), cmap='gray')
plt.title(f"Predicted: {predicted_label}, Actual:
{actual_label}") plt.show()

# Save the model
model.save("mnist_model.h5")

```

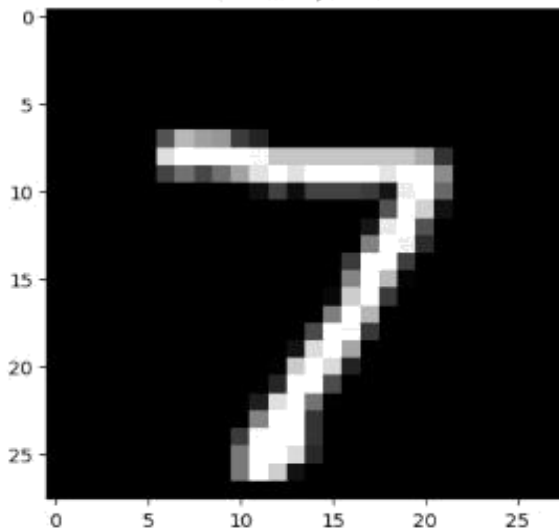
OUTPUT

```

Epoch 1/10
1500/1500 ————— 11s 6ms/step - accuracy: 0.8570 - loss: 0.4816 - val_accuracy: 0.9567 - val_loss: 0.1477
Epoch 2/10
1500/1500 ————— 10s 7ms/step - accuracy: 0.9637 - loss: 0.1225 - val_accuracy: 0.9664 - val_loss: 0.1154
Epoch 3/10
1500/1500 ————— 9s 6ms/step - accuracy: 0.9753 - loss: 0.0817 - val_accuracy: 0.9688 - val_loss: 0.1007
Epoch 4/10
1500/1500 ————— 12s 7ms/step - accuracy: 0.9832 - loss: 0.0537 - val_accuracy: 0.9732 - val_loss: 0.0930
Epoch 5/10
1500/1500 ————— 9s 6ms/step - accuracy: 0.9864 - loss: 0.0426 - val_accuracy: 0.9697 - val_loss: 0.0976
Epoch 6/10
1500/1500 ————— 10s 6ms/step - accuracy: 0.9907 - loss: 0.0303 - val_accuracy: 0.9727 - val_loss: 0.1007
Epoch 7/10
1500/1500 ————— 9s 6ms/step - accuracy: 0.9911 - loss: 0.0284 - val_accuracy: 0.9739 - val_loss: 0.1005
Epoch 8/10
1500/1500 ————— 10s 6ms/step - accuracy: 0.9924 - loss: 0.0215 - val_accuracy: 0.9719 - val_loss: 0.1215
Epoch 9/10
1500/1500 ————— 9s 5ms/step - accuracy: 0.9933 - loss: 0.0211 - val_accuracy: 0.9766 - val_loss: 0.1086
Epoch 10/10
1500/1500 ————— 10s 5ms/step - accuracy: 0.9938 - loss: 0.0175 - val_accuracy: 0.9738 - val_loss: 0.1121
313/313 ————— 1s 3ms/step - accuracy: 0.9739 - loss: 0.1211
Test Accuracy: 0.9769
313/313 ————— 1s 3ms/step

```

Predicted: 7, Actual: 7



as  Terminal

2. Deep Feed forward Networks

Objective: Implement a multilayer perceptron (MLP) and train it using the back-propagation algorithm.

Tasks: Train an MLP on the XOR problem, and demonstrate gradient-based learning and the use of hidden units.

```
# Import necessary libraries
import numpy as np

# Define activation function (Sigmoid) and its
derivative def sigmoid(x):
    return 1 / (1 + np.exp(-x))
def sigmoid_derivative(x):
    return x * (1 - x) # Derivative of sigmoid for backpropagation

# Define XOR input and expected output

y = np.array([[0], [1], [1], [0]]) # Expected outputs
# Set random seed for consistent
results np.random.seed(0)

# Initialize weights randomly using uniform distribution
W1 = np.random.uniform(-1, 1, (2, 4)) # Weights from Input (2) → Hidden Layer (4)
W2 = np.random.uniform(-1, 1, (4, 1)) # Weights from Hidden Layer (4) → Output (1)
lr = 0.1 # Learning rate

# Training the network (Backpropagation)
for epoch in range(20000): # Training for 20,000 epochs
    # Forward propagation
    hidden_layer = sigmoid(X @ W1)
    output_layer = sigmoid(hidden_layer @ W2)

    # Compute error
    error = y - output_layer

    # Backpropagation
    d_output = error * sigmoid_derivative(output_layer) # Output layer gradient
    d_hidden = (d_output @ W2.T) * sigmoid_derivative(hidden_layer) # Hidden layer
    gradient

    # Update weights
    W2 += hidden_layer.T @ d_output * lr
    W1 += X.T @ d_hidden * lr

    # Print loss every 5000 epochs
    if epoch % 5000 == 0:
        print(f'Epoch {epoch}, Loss: {np.mean(np.abs(error)):.4f} ')

# Print final predictions
print("\nFinal Output after Training:")
print(output_layer.round(3)) # Round values for better readability
```

OUTPUT

```
Epoch 0, Loss: 0.5001  
Epoch 5000, Loss: 0.4999  
Epoch 10000, Loss: 0.4980  
Epoch 15000, Loss: 0.1295
```

Final Output after Training:

```
[[0.094]  
 [0.929]  
 [0.929]  
 [0.054]]
```

3. Convolution Neural Networks (CNN) Basics

Objective: Implement a simple CNN for image classification.

Tasks: Write a Python program to build a CNN for the MNIST dataset, including convolution and pooling layers.

```
# Import necessary libraries
import tensorflow as tf
from tensorflow.keras import layers,
models import matplotlib.pyplot as plt
# Load the MNIST dataset
mnist = tf.keras.datasets.mnist
(x_train, y_train), (x_test, y_test) = mnist.load_data ()

# Normalize pixel values to [0,1]
x_train, x_test = x_train / 255.0, x_test / 255.0

# Reshape the dataset to add channel dimension (needed for CNN)
x_train = x_train.reshape(-1, 28, 28, 1)
x_test = x_test.reshape (-1, 28, 28, 1)

# Define CNN Model
model = models.Sequential([
    layers.Conv2D (32, (3,3), activation='relu', input_shape=(28,28,1)), # Convolution
    layers.MaxPooling2D((2,2)), # Pooling layer
    layers.Conv2D (64, (3,3), activation='relu'),
    layers.MaxPooling2D ((2,2)),
    layers.Conv2D (64, (3,3), activation='relu'),
    layers.Flatten(), # Flatten for dense layers
    layers.Dense (64, activation='relu'), # Fully connected layer
    layers.Dense(10, activation='softmax') # Output layer for 10 classes (digits 0-9)
])

# Compile the model
model.compile (optimizer='adam',
               loss='sparse_categorical_crossentropy',
               metrics=['accuracy'])

# Train the model
model.fit (x_train, y_train, epochs=5, validation_data=(x_test, y_test))

# Evaluate the model
test_loss, test_acc = model.evaluate (x_test, y_test)
print(f"Test accuracy: {test_acc:.4f}")

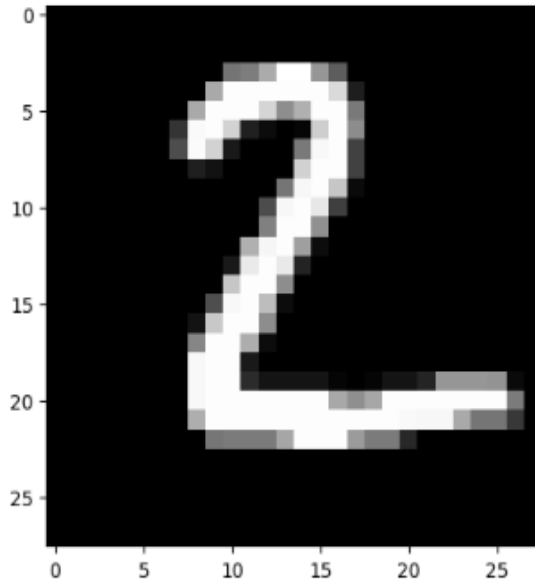
# Make a prediction on a test image
predictions = model.predict(x_test[:5])

# Display the first test image with its predicted label
plt.imshow(x_test[1].reshape(28, 28), cmap="gray")
plt.title(f"Predicted Label: {predictions[1].argmax()}")
plt.show()
```

OUTPUT

```
Epoch 1/5  
1875/1875 — 53s 27ms/step - accuracy: 0.8937 - loss: 0.3417 - val_accuracy: 0.9851 - val_loss: 0.0453  
Epoch 2/5  
1875/1875 — 51s 27ms/step - accuracy: 0.9854 - loss: 0.0478 - val_accuracy: 0.9841 - val_loss: 0.0474  
Epoch 3/5  
1875/1875 — 84s 28ms/step - accuracy: 0.9893 - loss: 0.0344 - val_accuracy: 0.9895 - val_loss: 0.0333  
Epoch 4/5  
1875/1875 — 79s 27ms/step - accuracy: 0.9920 - loss: 0.0241 - val_accuracy: 0.9907 - val_loss: 0.0283  
Epoch 5/5  
1875/1875 — 82s 27ms/step - accuracy: 0.9939 - loss: 0.0206 - val_accuracy: 0.9913 - val_loss: 0.0293  
313/313 — 3s 8ms/step - accuracy: 0.9902 - loss: 0.0336  
Test accuracy: 0.9913  
1/1 — 0s 110ms/step
```

Predicted Label: 2



4. Advanced Convolutional Networks

Objective: Explore different variants of convolution functions and pooling operations.

Tasks: Implement and compare different CNN architectures using various convolution and pooling techniques. Experiment with efficient convolution algorithms and random features.

```
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers, models
import numpy as np
import matplotlib.pyplot as plt

# Load CIFAR-10 dataset
(x_train, y_train), (x_test, y_test) = keras.datasets.cifar10.load_data()
x_train, x_test = x_train / 255.0, x_test / 255.0 # Normalize

# Define different CNN architectures

# 1. Standard CNN
def standard_cnn():
    model = models.Sequential([
        layers.Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(32, 32, 3)),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.GlobalAveragePooling2D(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

# 2. CNN with Depthwise Separable
# Convolutions
def depthwise_separable_cnn():
    model = models.Sequential([
        layers.SeparableConv2D(32, (3, 3), activation='relu', padding='same',
input_shape=(32, 32, 3)),
        layers.MaxPooling2D((2, 2)),
        layers.SeparableConv2D(64, (3, 3), activation='relu',
padding='same'), layers.MaxPooling2D((2, 2)),
        layers.SeparableConv2D(128, (3, 3), activation='relu',
padding='same'), layers.GlobalAveragePooling2D(),
        layers.Dense(64, activation='relu'),
        layers.Dense(10, activation='softmax')
    ])
    return model

# 3. CNN with Dilated Convolutions
def dilated_cnn():
    model = models.Sequential([
        layers.Conv2D(32, (3, 3), activation='relu', padding='same',
dilation_rate=2, input_shape=(32, 32, 3)),
```

```

layers.MaxPooling2D((2, 2)),
layers.Conv2D(64, (3, 3), activation='relu', padding='same',
dilation_rate=2), layers.MaxPooling2D((2, 2)),
layers.Conv2D(128, (3, 3), activation='relu', padding='same',
dilation_rate=2), layers.GlobalAveragePooling2D(),
layers.Dense(64, activation='relu'),
layers.Dense(10, activation='softmax')
])
return model

# 4. CNN with Random Feature Projections (Random
Convolutions) def random_features_cnn():
    model = models.Sequential([
        layers.Conv2D(32, (3, 3), activation='relu', padding='same', input_shape=(32, 32, 3)),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(64, (3, 3), activation='relu', padding='same'),
        layers.MaxPooling2D((2, 2)),
        layers.Conv2D(128, (3, 3), activation='relu', padding='same'),
        layers.GlobalAveragePooling2D(),
        layers.Dense(64, activation='relu', kernel_initializer='random_normal'), # Random
initialization
        layers.Dense(10, activation='softmax')
    ])
    return model

# Training function
def train_and_evaluate(model, name):
    model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
metrics=['accuracy'])
    print(f"\nTraining {name} model...")
    history = model.fit(x_train, y_train, epochs=10, batch_size=64, validation_data=(x_test,
y_test))
    test_loss, test_acc = model.evaluate(x_test, y_test, verbose=2)
    print(f'{name} Test accuracy: {test_acc:.4f}')
    return history

# Train and compare
models_dict = {
    "Standard CNN": standard_cnn(),
    "Depthwise CNN": depthwise_separable_cnn(),
    "Dilated CNN": dilated_cnn(),
    "Random Feature CNN": random_features_cnn()
}

histories = {}
for name, model in models_dict.items():
    histories[name] = train_and_evaluate(model, name)

# Plot accuracy comparison
plt.figure(figsize=(10, 5))
for name, history in histories.items():

```

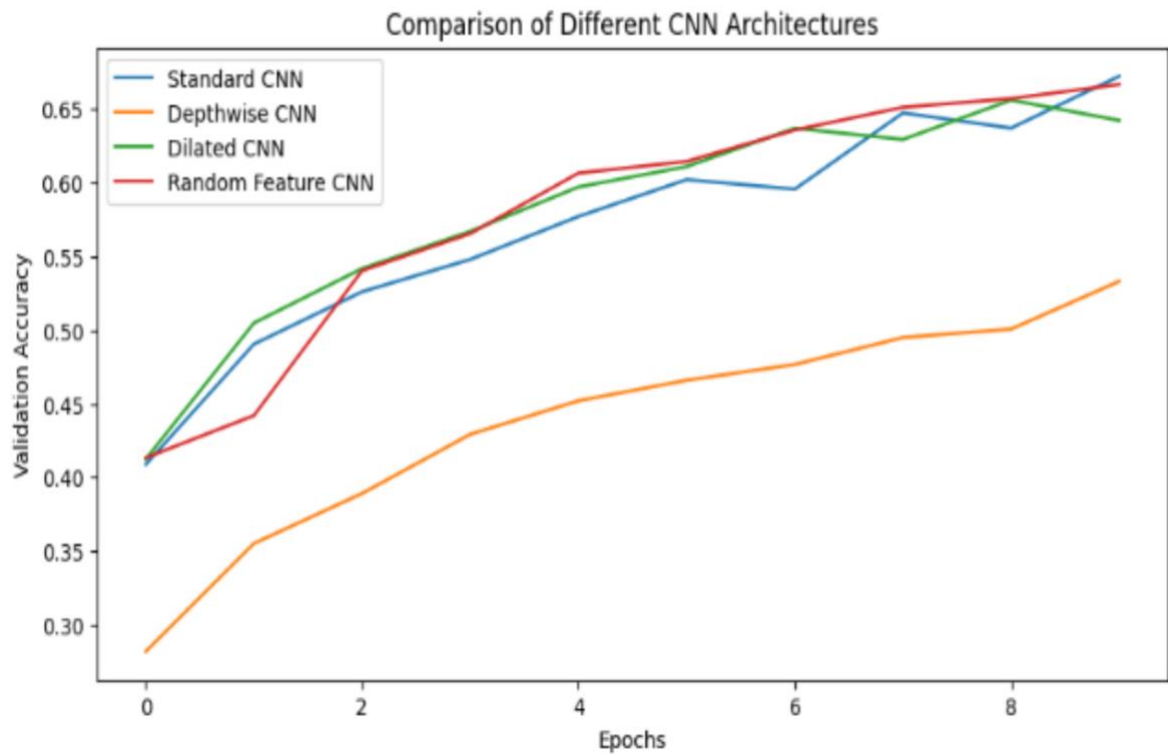
```
plt.plot(history.history['val_accuracy'], label=name)

plt.xlabel("Epochs")
plt.ylabel("Validation Accuracy")
plt.legend()
plt.title("Comparison of Different CNN Architectures")
plt.show()
```

OUTPUT

Training Random Feature CNN model...

```
Epoch 1/10
782/782 ————— 122s 154ms/step - accuracy: 0.2435 - loss: 1.9796 - val_accuracy: 0.4134 - val_loss: 1.6102
Epoch 2/10
782/782 ————— 146s 159ms/step - accuracy: 0.4252 - loss: 1.5566 - val_accuracy: 0.4420 - val_loss: 1.4953
Epoch 3/10
782/782 ————— 142s 159ms/step - accuracy: 0.4914 - loss: 1.3844 - val_accuracy: 0.5402 - val_loss: 1.2601
Epoch 4/10
782/782 ————— 137s 153ms/step - accuracy: 0.5420 - loss: 1.2709 - val_accuracy: 0.5653 - val_loss: 1.1984
Epoch 5/10
782/782 ————— 142s 153ms/step - accuracy: 0.5704 - loss: 1.1876 - val_accuracy: 0.6063 - val_loss: 1.1254
Epoch 6/10
782/782 ————— 146s 159ms/step - accuracy: 0.5954 - loss: 1.1264 - val_accuracy: 0.6142 - val_loss: 1.0734
Epoch 7/10
782/782 ————— 138s 154ms/step - accuracy: 0.6254 - loss: 1.0497 - val_accuracy: 0.6355 - val_loss: 1.0145
Epoch 8/10
782/782 ————— 119s 152ms/step - accuracy: 0.6413 - loss: 1.0033 - val_accuracy: 0.6509 - val_loss: 0.9881
Epoch 9/10
782/782 ————— 142s 153ms/step - accuracy: 0.6539 - loss: 0.9679 - val_accuracy: 0.6568 - val_loss: 0.9707
Epoch 10/10
782/782 ————— 146s 158ms/step - accuracy: 0.6739 - loss: 0.9211 - val_accuracy: 0.6664 - val_loss: 0.9281
313/313 - 7s - 23ms/step - accuracy: 0.6664 - loss: 0.9281
Random Feature CNN Test accuracy: 0.6664
```



5. Recurrent Neural Networks (RNN)

Objective: Implement and train a basic RNN for sequence modeling.

Tasks: Write a Python program to build an RNN for text generation or sequence prediction, using unfolding computational graphs.

```
# Install TensorFlow (if not already
installed) !pip install tensorflow

# Import required libraries
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import SimpleRNN, Dense
import matplotlib.pyplot as plt

# Function to generate synthetic sequential data
def generate_sequence(seq_length=5, num_samples=1000):
    X, y = [], []
    for _ in range(num_samples):
        seq = np.random.rand(seq_length)
        X.append(seq)
        y.append(seq[-1] + 0.1) # Predict next value with a small shift
    return np.array(X), np.array(y)

# Define sequence length
seq_length = 5

# Generate dataset
X, y = generate_sequence(seq_length)

# Reshape data for RNN input: (samples, timesteps,
features) X = X.reshape((X.shape[0], seq_length, 1))

# Split data into training and testing sets (80% train, 20%
test) split = int(0.8 * len(X))
X_train, y_train = X[:split], y[:split]
X_test, y_test = X[split:], y[split:]

# Build a simple RNN model
model = Sequential([
    SimpleRNN(10, activation='relu', input_shape=(seq_length, 1)), # RNN Layer
    Dense(1) # Output layer
])

# Compile the model
model.compile(optimizer='adam', loss='mse')

# Display model summary
model.summary()

# Train the model
```

```
history = model.fit(X_train, y_train, epochs=50, batch_size=16, validation_data=(X_test,
y_test))
```

```
# Evaluate the model
```

```
test_loss = model.evaluate(X_test, y_test)
```

```
print(f"Test Loss: {test_loss}")
```

```
# Predict on a new sample input
```

```
sample_input = np.array([0.1, 0.2, 0.3, 0.4, 0.5]).reshape(1, seq_length, 1)
```

```
predicted_value = model.predict(sample_input)
```

```
print(f"Predicted next value: {predicted_value}")
```

```
# Plot training loss vs validation loss
```

```
plt.plot(history.history['loss'], label='Train Loss')
```

```
plt.plot(history.history['val_loss'], label='Validation  
Loss') plt.xlabel('Epochs')
```

```
plt.ylabel('Loss')
```

```
plt.legend()
```

```
plt.title('Training Loss vs Validation  
Loss') plt.show()
```

OUTPUT

Layer (type)	Output Shape	Param #
simple_rnn_1 (SimpleRNN)	(None, 10)	120
dense_1 (Dense)	(None, 1)	11

Total params: 131 (524.00 B)

Trainable params: 131 (524.00 B)

Non-trainable params: 0 (0.00 B)

Epoch 1/50

50/50 ————— 4s 24ms/step - loss: 0.0757 - val_loss: 0.0130

Epoch 2/50

50/50 ————— 0s 6ms/step - loss: 0.0075 - val_loss: 0.0024

Epoch 3/50

50/50 ————— 1s 8ms/step - loss: 0.0021 - val_loss: 0.0014

Epoch 4/50

50/50 ————— 0s 4ms/step - loss: 0.0018 - val_loss: 0.0010

Epoch 5/50

50/50 ————— 0s 4ms/step - loss: 0.0010 - val_loss: 7.5331e-04

Epoch 6/50

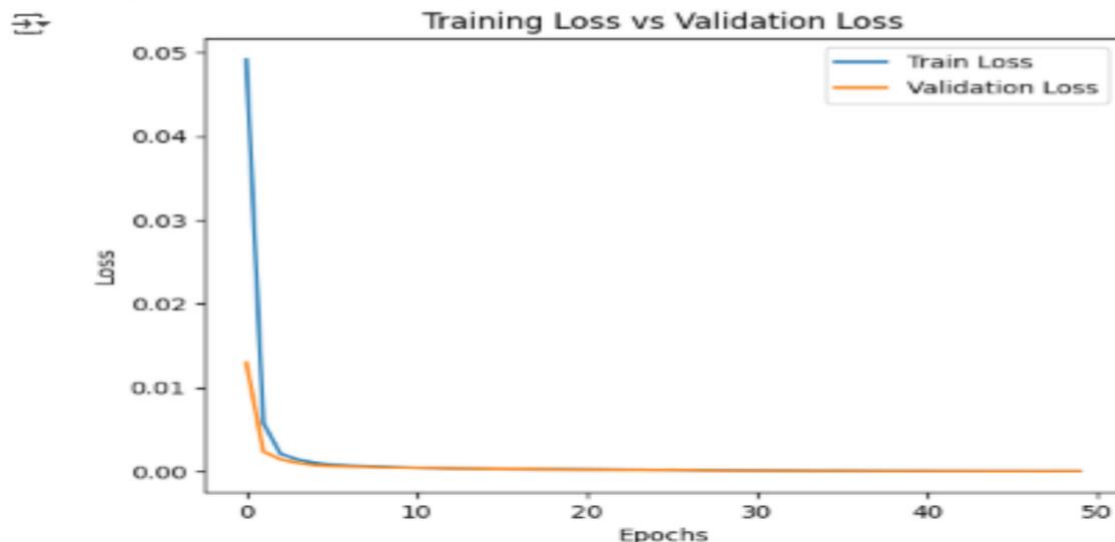
50/50 ————— 0s 4ms/step - loss: 8.4793e-04 - val_loss: 6.3633e-04

Epoch 7/50

50/50 ————— 0s 4ms/step - loss: 6.4328e-04 - val_loss: 5.7732e-04

Epoch 8/50

Predicted next value: [[0.6039668]]



6. Sequence-to-Sequence Models

Objective: Implement encoder-decoder architecture for sequence-to-sequence tasks.

Tasks: Train a sequence-to-sequence model for machine translation or text summarization using bidirectional RNNs and deep recurrent networks.

```
import numpy as np
import tensorflow as tf
import unicodedata
from tensorflow.keras.layers import Input, LSTM, Dense
from tensorflow.keras.models import Model

# -----
# 1. Normalize Function
# -----
def normalize(text):
    return unicodedata.normalize('NFC', text)

# -----
# 2. Data
# -----
data = [("hello", "नमस्ते"), ("how are you", "आप कैसे हैं"),
        ("i am fine", "मैं ठीक हूँ"), ("thank you", "धन्यवाद"),
        ("good night", "शुभ रात्रि")]

inp_texts, tgt_texts = [], []
inp_chars, tgt_chars = set(), set()

for i, t in data:
    i = normalize(i)
    t = normalize(t)
    t = '\t' + t + '\n'

    inp_texts.append(i)
    tgt_texts.append(t)

    inp_chars.update(i)
    tgt_chars.update(t)

# -----
# 3. Vocabulary
# -----
inp_chars = sorted(inp_chars)
tgt_chars = sorted(tgt_chars)

= {c:i for i,c in enumerate(inp_chars)}
tg_tok = {c:i for i,c in enumerate(tgt_chars)}
```

```

rev_tg = {i:c for c,i in tg_tok.items()}

max_in = max(len(x) for x in inp_texts)
max_tg = max(len(x) for x in tgt_texts)

# -----
# 4. Vectorization
# -----
enc_in = np.zeros((len(inp_texts), max_in,
len(inp_chars)))
dec_in = np.zeros((len(inp_texts), max_tg,
len(tgt_chars)))
dec_tar = np.zeros_like(dec_in)

for i,(inp,tgt) in enumerate(zip(inp_texts,tgt_texts)):
    for t,c in enumerate(inp):
        enc_in[i,t,in_tok[c]] = 1
    for t,c in enumerate(tgt):
        dec_in[i,t,tg_tok[c]] = 1
        if t>0:
            dec_tar[i,t-1,tg_tok[c]] = 1

# -----
# 5. Encoder
# -----
latent = 256

enc_inputs = Input(shape=(None,len(inp_chars)))
_, h, c = LSTM(latent, return_state=True)(enc_inputs)
enc_states = [h, c]

# -----
# 6. Decoder
# -----
dec_inputs = Input(shape=(None,len(tgt_chars)))
dec_lstm = LSTM(latent, return_sequences=True,
return_state=True)
dec_out, _, _ = dec_lstm(dec_inputs,
initial_state=enc_states)

dense = Dense(len(tgt_chars), activation='softmax')
dec_out = dense(dec_out)

model = Model([enc_inputs, dec_inputs], dec_out)
model.compile(optimizer='adam',
loss='categorical_crossentropy',metrics=['accuracy'])
model.fit([enc_in, dec_in], dec_tar, epochs=300,
verbose=0)
model.summary()

```

```

# -----
# 7. Inference Models
# -----
enc_model = Model(enc_inputs, enc_states)

h_in = Input(shape=(latent,))
c_in = Input(shape=(latent,))

dec_out2, h, c = dec_lstm(dec_inputs,
initial_state=[h_in, c_in])
dec_out2 = dense(dec_out2)

dec_model = Model(
    [dec_inputs, h_in, c_in],
    [dec_out2, h, c]
)

# -----
# 8. Decode
# -----
def decode(seq):
    states = enc_model.predict(seq, verbose=0)

    target = np.zeros((1,1,len(tgt_chars)))
    target[0,0,tg_tok['\t']] = 1

    result = ""
    while True:
        pred, h, c = dec_model.predict([target] + states,
verbose=0)
        idx = np.argmax(pred[0,-1])
        char = rev_tg[idx]

        if char == '\n' or len(result) > max_tg:
            break

        result += char

        target = np.zeros((1,1,len(tgt_chars)))
        target[0,0,idx] = 1
        states = [h, c]

    return result

```

```

# -----
# 9. Test
# -----
for i in range(len(inp_texts)):
    print(inp_texts[i], "->", decode(enc_in[i:i+1]))

```

OUTPUT :

Model: "functional"

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, None, 18)	0	-
input_layer_1 (InputLayer)	(None, None, 29)	0	-
lstm (LSTM)	[(None, 256), (None, 256), (None, 256)]	281,600	input_layer[0][0]
lstm_1 (LSTM)	[(None, None, 256), (None, 256), (None, 256)]	292,864	input_layer_1[0]... lstm[0][1], lstm[0][2]
dense (Dense)	(None, None, 29)	7,453	lstm_1[0][0]

Total params: 1,745,753 (6.66 MB)
 Trainable params: 581,917 (2.22 MB)
 Non-trainable params: 0 (0.00 B)
 Optimizer params: 1,163,836 (4.44 MB)

hello -> नमस्ते
 how are you -> आप कैसे हैं
 i am fine -> मैं ठीक हूँ
 thank you -> धन्यवाद
 good night -> शुभ रात्रि

7. Long Short-Term Memory (LSTM) Networks

Objective: Implement LSTM networks and explore their benefits over standard RNNs.

Tasks: Build and train an LSTM network for a time series prediction problem, comparing it with Echo State Networks and other gated RNNs.

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.metrics import mean_squared_error
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

# ✓ Echo State Network (ESN)
implementation class ESN:
    def __init__(self, n_inputs, n_outputs, n_reservoir=100,
spectral_radius=0.95, sparsity=0.1, random_state=None):
        self.n_inputs = n_inputs
        self.n_outputs = n_outputs
        self.n_reservoir = n_reservoir
        self.spectral_radius = spectral_radius
        self.sparsity = sparsity
        self.random_state = np.random.RandomState(random_state)

        self._Win = self.random_state.rand(self.n_reservoir, self.n_inputs) - 0.5
        W = self.random_state.rand(self.n_reservoir, self.n_reservoir) - 0.5
        # Apply sparsity
        W[self.random_state.rand(*W.shape) > self.sparsity] = 0
        # Spectral radius adjustment
        radius = np.max(np.abs(np.linalg.eigvals(W)))
        self._W = W * (self.spectral_radius / radius)

    def _update(self, state, input_signal):
        pre_activation = np.dot(self._Win, input_signal) + np.dot(self._W, state)
        return np.tanh(pre_activation)

    def fit(self, X, y):
        states = np.zeros((X.shape[0], self.n_reservoir))
        state = np.zeros(self.n_reservoir)
        for t in range(X.shape[0]):
            state = self._update(state, X[t])
            states[t] = state
        # Ridge regression (simple)
        self._Wout = np.dot(np.linalg.pinv(states), y)

    def predict(self, X):
        predictions = []
        state = np.zeros(self.n_reservoir)
        for t in range(X.shape[0]):
            state = self._update(state, X[t])
            y = np.dot(self._Wout, state)
            predictions.append(y)
        return np.array(predictions)
```



```

# 1. Generate sine wave data
time = np.arange(0, 100,
0.1) series = np.sin(time)

# 2. Prepare dataset using sliding window
def create_dataset(data, window=20):
    X, y = [], []
    for i in range(len(data) - window):
        X.append(data[i:i+window])
        y.append(data[i+window])
    return np.array(X), np.array(y)

window_size = 20
X, y = create_dataset(series, window=window_size)
X = X.reshape((X.shape[0], window_size, 1))

# Split into training and testing
setsplit = int(0.8 * len(X))
X_train, X_test = X[:split], X[split:]
y_train, y_test = y[:split], y[split:]

# 3. LSTM model
lstm = Sequential([
    LSTM(50, input_shape=(window_size, 1)),
    Dense(1)
])
lstm.compile(optimizer='adam', loss='mse')
lstm.fit(X_train, y_train, epochs=10, batch_size=32, verbose=0)

# Predict with LSTM
lstm_pred = lstm.predict(X_test).flatten()

# 4. ESN model (reshape input for ESN)
X_esn = X.reshape(X.shape[0], window_size)
X_train_esn, X_test_esn = X_esn[:split], X_esn[split:]

esn = ESN(n_inputs=window_size, n_outputs=1, n_reservoir=100, random_state=42)
esn.fit(X_train_esn, y_train)
esn_pred = esn.predict(X_test_esn)

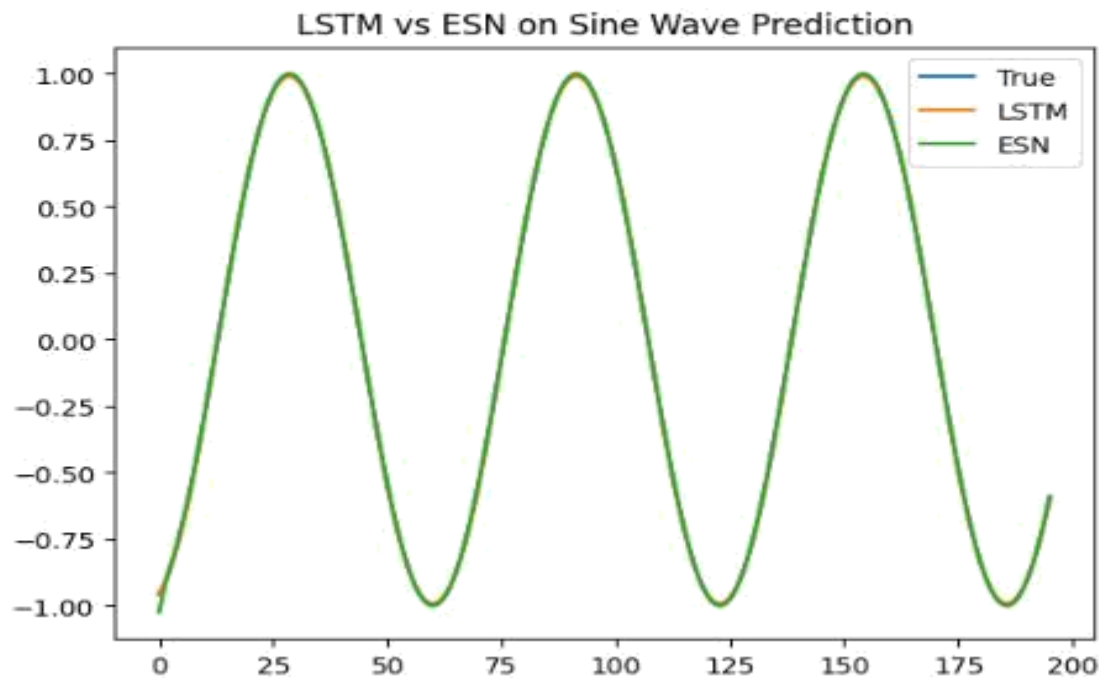
# 5. Compare and plot
print("LSTM MSE:", mean_squared_error(y_test, lstm_pred))
print("ESN MSE:", mean_squared_error(y_test, esn_pred))

plt.plot(y_test, label='True')
plt.plot(lstm_pred, label='LSTM')
plt.plot(esn_pred, label='ESN')
plt.legend()
plt.title("LSTM vs ESN on Sine Wave Prediction")
plt.show()

```

OUTPUT

```
↔ /usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: 1  
  super().__init__(**kwargs)  
7/7 ————— 0s 36ms/step  
LSTM MSE: 4.304803138162043e-05  
ESN MSE: 2.5677328845235095e-05
```



8. Autoencoders

Objective: Implement different types of autoencoders.

Tasks: Write Python programs to build and train undercomplete, denoising, and variational autoencoders on image data. Explore regularization techniques and representational power.

```
import numpy as np, tensorflow as tf
from tensorflow.keras import layers, models
import matplotlib.pyplot as plt

# Load & preprocess MNIST
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
x_train, x_test = x_train.astype("float32") / 255., x_test.astype("float32") / 255.
x_train, x_test = x_train[..., np.newaxis], x_test[..., np.newaxis]

# Add noise for Denoising Autoencoder
def add_noise(x): return np.clip(x + 0.5 * np.random.randn(*x.shape), 0., 1.)
x_train_noisy, x_test_noisy = add_noise(x_train), add_noise(x_test)

# Common encoder and decoder layers
enc = models.Sequential([layers.Input((28, 28, 1)), layers.Flatten(),
layers.Dense(64, activation='relu')])
dec = models.Sequential([layers.Input((64,)), layers.Dense(784,
activation='sigmoid'), layers.Reshape((28, 28, 1))])

# 1 Undercomplete Autoencoder ucae
= models.Sequential([enc, dec])
ucaec.compile(optimizer='adam', loss='binary_crossentropy')
ucaec.fit(x_train, x_train, epochs=5, batch_size=256, verbose=1)

# 2 Denoising Autoencoder
dnaec = models.Sequential([enc, dec])
dnaec.compile(optimizer='adam', loss='binary_crossentropy')
dnaec.fit(x_train_noisy, x_train, epochs=5, batch_size=256, verbose=1)

# 3 Variational Autoencoder (fixed)
class Sampling(layers.Layer):
    def call(self, inputs): z_mean,
        z_log_var = inputs
        epsilon = tf.random.normal(shape=tf.shape(z_mean))
        return z_mean + tf.exp(0.5 * z_log_var) * epsilon

# Encoder for VAE
inputs = layers.Input(shape=(28, 28, 1))
x = layers.Flatten()(inputs)
x = layers.Dense(128, activation="relu")(x)
z_mean = layers.Dense(2, name="z_mean")(x)
z_log_var = layers.Dense(2, name="z_log_var")(x)
z = Sampling()([z_mean, z_log_var])
encoder = tf.keras.Model(inputs, [z_mean, z_log_var, z], name="encoder")

# Decoder for VAE
latent_inputs = layers.Input(shape=(2,))
x = layers.Dense(128, activation="relu")(latent_inputs)
```

```
x = layers.Dense(784, activation="sigmoid")(x)
outputs = layers.Reshape((28, 28, 1))(x)
decoder = tf.keras.Model(latent_inputs, outputs, name="decoder")
```

```
# Custom VAE class
```

```
class VAE(tf.keras.Model):
```

```
    def __init__(self, encoder, decoder, **kwargs):
```

```
        super().__init__(**kwargs)
```

```
        self.encoder = encoder
```

```
        self.decoder = decoder
```

```
    def call(self, inputs):
```

```
        z_mean, z_log_var, z = self.encoder(inputs)
```

```
        reconstruction = self.decoder(z)
```

```
        # Compute loss inside the model (safe with symbolic
```

```
        tensors) recon_loss = tf.reduce_mean(
```

```
            tf.keras.losses.binary_crossentropy(inputs,
```

```
            reconstruction) ) * 784
```

```
        kl_loss = -0.5 * tf.reduce_mean(
```

```
            1 + z_log_var - tf.square(z_mean) - tf.exp(z_log_var)
```

```
        )
```

```
        self.add_loss(recon_loss + kl_loss)
```

```
        return reconstruction
```

```
# Compile and train VAE vae =
```

```
VAE(encoder, decoder)
```

```
vae.compile(optimizer="adam")
```

```
vae.fit(x_train, x_train, epochs=5, batch_size=256, verbose=1)
```

```
# Plotting function
```

```
def show_imgs(orig, recon, title):
```

```
    plt.figure(figsize=(10, 2))
```

```
    n = min(10, len(orig))
```

```
    for i in range(n):
```

```
        plt.subplot(2, n, i+1)
```

```
        img = orig[i].squeeze() if orig[i].ndim == 3 else
```

```
        orig[i] plt.imshow(img, cmap='gray')
```

```
        plt.axis('off')
```

```
        plt.subplot(2, n, i + n + 1)
```

```
        img = recon[i].squeeze() if recon[i].ndim == 3 else
```

```
        recon[i] plt.imshow(img, cmap='gray')
```

```
        plt.axis('off')
```

```
    plt.suptitle(title)
```

```
    plt.tight_layout()
```

```
    plt.show()
```

```
# Visualize reconstructions
```

```
show_imgs(x_test, ucae.predict(x_test), "Undercomplete AE")
```

```
show_imgs(x_test, dnae.predict(x_test_noisy), "Denoising AE")
```

```
show_imgs(x_test, vae.predict(x_test), "VAE")
```

OUTPUT

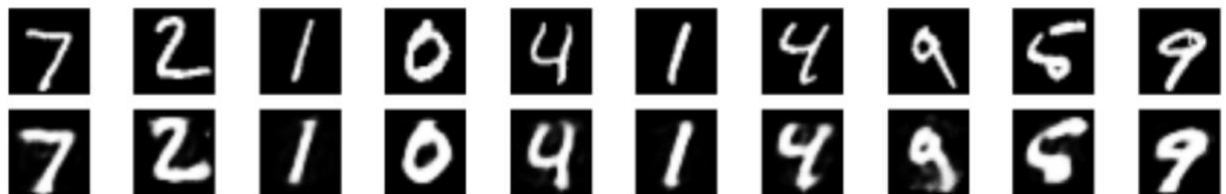
```

Epoch 1/5
235/235 ————— 3s 9ms/step - loss: 0.3453
Epoch 2/5
235/235 ————— 2s 9ms/step - loss: 0.1540
Epoch 3/5
235/235 ————— 2s 9ms/step - loss: 0.1237
Epoch 4/5
235/235 ————— 3s 9ms/step - loss: 0.1071
Epoch 5/5
235/235 ————— 3s 11ms/step - loss: 0.0971
Epoch 1/5
235/235 ————— 3s 8ms/step - loss: 0.1805
Epoch 2/5
235/235 ————— 2s 9ms/step - loss: 0.1247
Epoch 3/5
235/235 ————— 3s 9ms/step - loss: 0.1210
Epoch 4/5
235/235 ————— 3s 13ms/step - loss: 0.1189
Epoch 5/5
235/235 ————— 2s 10ms/step - loss: 0.1176
Epoch 1/5
235/235 ————— 4s 12ms/step - loss: 303.1105
Epoch 2/5
235/235 ————— 6s 16ms/step - loss: 187.0279
Epoch 3/5
235/235 ————— 4s 12ms/step - loss: 177.2749
Epoch 4/5
235/235 ————— 5s 11ms/step - loss: 170.5524
Epoch 5/5
235/235 ————— 4s 16ms/step - loss: 166.8546
313/313 ————— 1s 1ms/step

```



Undercomplete AE



313/313 ————— 1s 1ms/step

Denoising AE



313/313 ————— 1s 2ms/step

VAE

